



University of Antwerp

Faculty of Applied  
Engineering

# **4-Computerarchitectuur en besturingssystemen 2:** **OS**

Laatste aanpassing: 21/04/2026



University of Antwerp

Faculty of Applied  
Engineering

# Inleiding

# Doelstellingen

- Doel van het labo:
  - Beperkingen van een bare metal systeem ondervinden
  - Een embedded OS project kunnen opzetten
  - Multitasking concepten waaronder tasks, semaforen en queues kunnen toepassen
  - Inzicht verkrijgen in het geheugenbeheer onder een embedded OS
  - Realtime aspect



# Documentatie

- Powerpoint presentatie
- Labotekst
- FreeRTOS reference manual
- Mastering the FreeRTOS realtime kernel: a hands on tutorial guide
- Documentatie, software, PPT: zie blackboard



- Portfolio (20%)

In te dienen:

- Code projecten
- Antwoorden op gestelde vragen + metingen

- Laatste zitting: toets (80%)

- Op papier, zonder robot HW
- Open boek, echter zonder internet connectie

# Overzicht oefeningen

- Z1: Oef1: Parallele tasks zonder OS
- Z1: Oef2: Opzetten van een FreeRTOS project
- Z2: Oef3: Oef1 herhalen met FreeRTOS tasks
- Z2: Oef4: Dynamische geheugenallocatie
- Z3: Oef5: Stack variabelen, stack overflow detectie
- Z3: Oef6: Intertask communicatie
- Z4: Oef7: Communicatie tussen tasks: FIFO queue
- Z4: Oef8: Integratie van ISR's en ISR<->Task comms
- Z5: Oef10: Motor hoekpositie- en snelheidscontroller

Oef9 niet uitwerken



University of Antwerp

Faculty of Applied  
Engineering

# FreeRTOS inleiding

- Open source
- Multi platform
- Gemakkelijk porteerbaar
- Basisfeatures
  - Cooperative / pre-emptive multitasking
  - Met / zonder time slicing (tick)
  - Fixed priority (instelbaar)
- Omvat:
  - Geheugenbeheer
  - Scheduling
  - Intertask comms: semaphores, mutexen, queues, atomic operations





# Opzetten van een FreeRTOS project

- FreeRTOS directory bevat o.a.:

|                   |                                             |
|-------------------|---------------------------------------------|
| FreeRTOS\heap2.c  | FreeRTOS dynamische geheugen allocatie code |
| FreeRTOS\list.c   | FreeRTOS lijst implementatie                |
| FreeRTOS\port.c   | FreeRTOS processor specifieke code          |
| FreeRTOS\queue.c  | FreeRTOS queue code                         |
| FreeRTOS\tasks.c  | FreeRTOS tasks code                         |
| FreeRTOS\timers.c | FreeRTOS timer code                         |

- Configuratie file: FreeRTOSConfig.h
  - Processor frequentie
  - Tick rate
  - Aan / uit schakelen van features → geheugen, CPU besparing

- Hulpbestanden

|          |                                                          |
|----------|----------------------------------------------------------|
| hooks.c  | Foutafhandeling o.a. bij stack overflow, mem alloc error |
| memmap.c | Tool om een geheugenkaart op te maken                    |

- Start v/d scheduler: vTaskStartScheduler();





University of Antwerp

Faculty of Applied  
Engineering

# Tasks



# Task states

- **Running**: Task wordt momenteel uitgevoerd
- **Ready**: Task wordt momenteel niet uitgevoerd wegens een andere taak met gelijke of hogere prioriteit die uitgevoerd wordt. Klaar om uitgevoerd te worden.
- **Blocked**: Task wacht op een actie om in Ready/Running state te komen. Bv wachten op een timer / mutex / semafoor. Heeft altijd een timeout waarna de task ready / running wordt.
- **Suspended**: Idem als blocked, echter zonder timeout



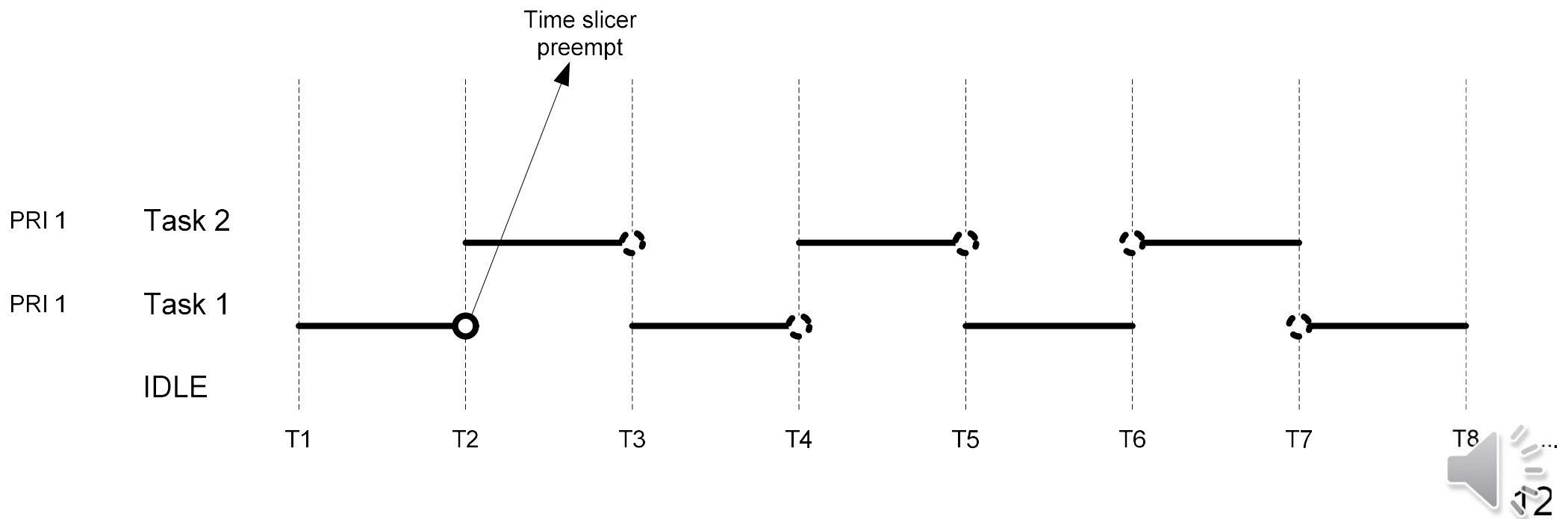
# Scheduling voorbeelden (1)

Task 1:

```
while (1)
{
    ...
}
```

Task 2:

```
while (1)
{
    ...
}
```



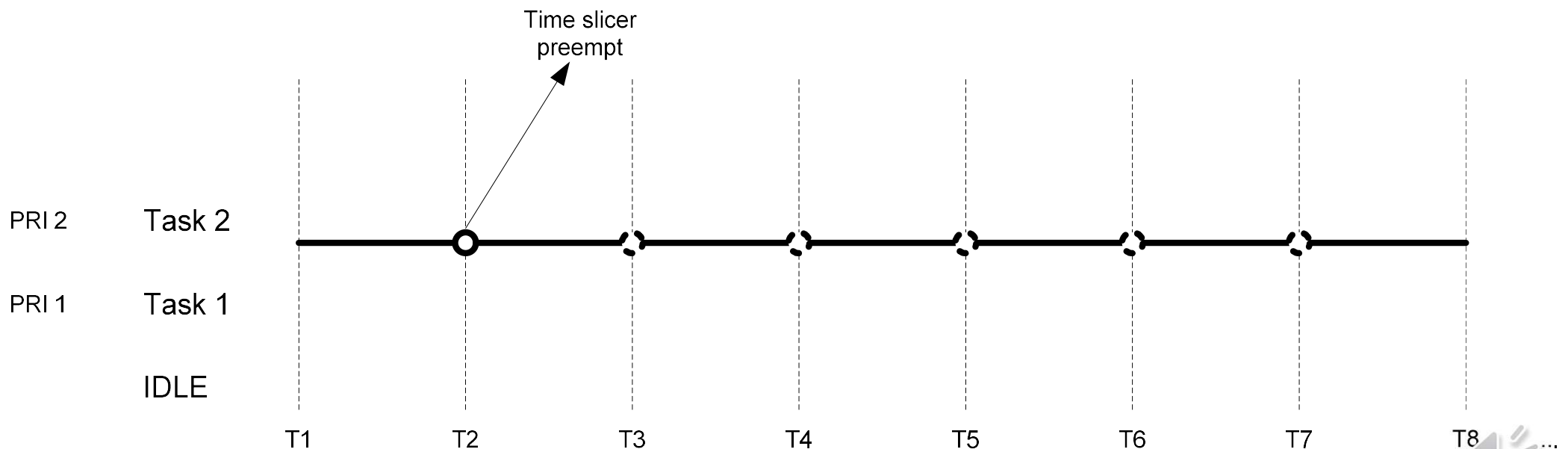
# Scheduling voorbeelden (2)

Task 1:

```
while (1)
{
    ...
}
```

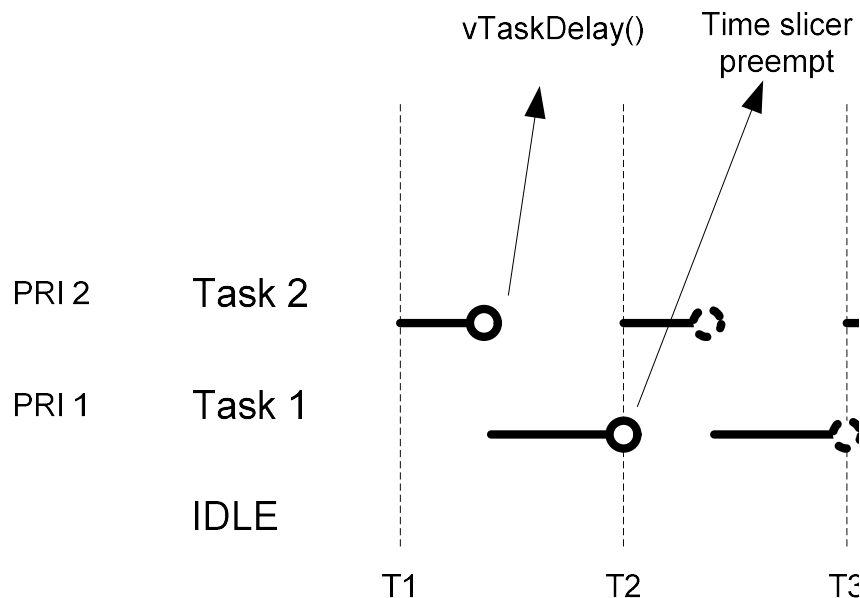
Task 2:

```
while (1)
{
    ...
}
```



# Sche

```
Task 1:
while (1)
{
    ...
}
```



## 2.9 vTaskDelay()

```
#include "FreeRTOS.h"
#include "task.h"

void vTaskDelay( TickType_t xTicksToDelay );
```

Listing 20 vTaskDelay() function prototype

### Summary

Places the task that calls vTaskDelay() into the Blocked state for a fixed number of tick interrupts.

Specifying a delay period of zero ticks will not result in the calling task being placed into the Blocked state, but will result in the calling task yielding to any Ready state tasks that share its priority. Calling vTaskDelay( 0 ) is equivalent to calling taskYIELD().

### Parameters

**xTicksToDelay** The number of tick interrupts that the calling task will remain in the Blocked state before being transitioned back into the Ready state. For example, if a task called vTaskDelay( 100 ) when the tick count was 10,000, then it would immediately enter the Blocked state and remain in the Blocked state until the tick count reached 10,100.

Any time that remains between vTaskDelay() being called, and the next tick interrupt occurring, counts as one complete tick period. Therefore, the highest time resolution that can be achieved when specifying a delay period is, in the worst case, equal to one complete tick interrupt period.

The macro pdMS\_TO\_TICKS() can be used to convert milliseconds into ticks. This is demonstrated in the example in this section.

### Return Values

None.

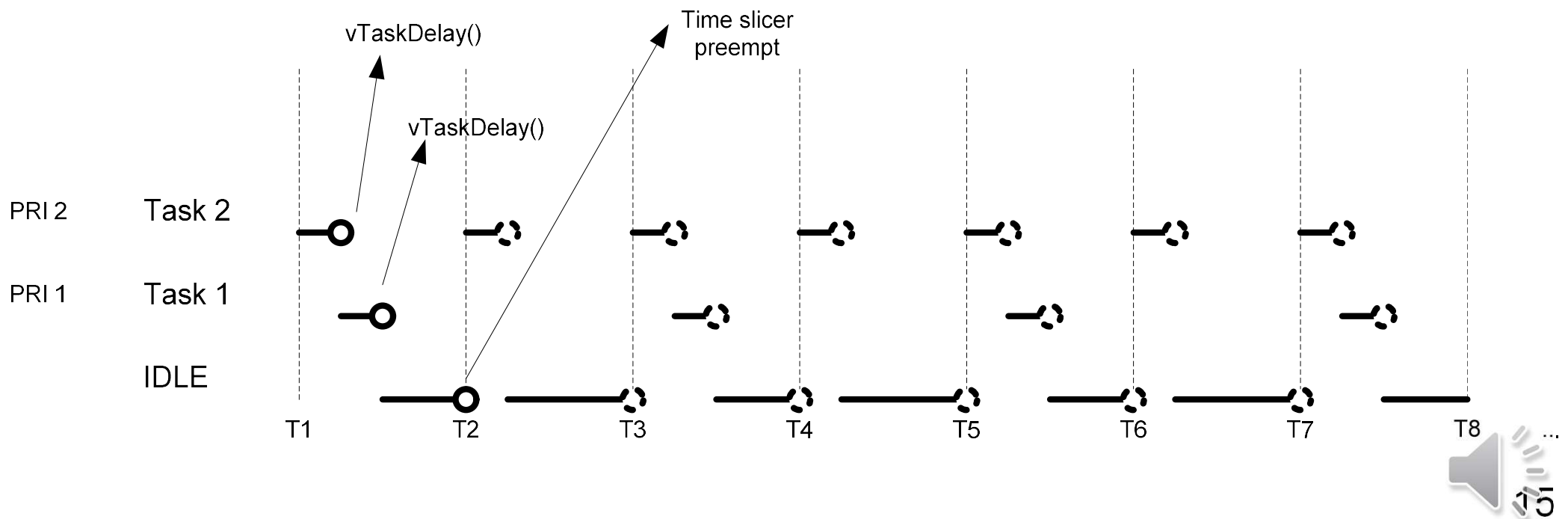
# Scheduling voorbeelden (4)

Task 1:

```
while (1)
{
    ...
    vTaskDelay(2)
}
```

Task 2:

```
while (1)
{
    ...
    vTaskDelay(1)
}
```



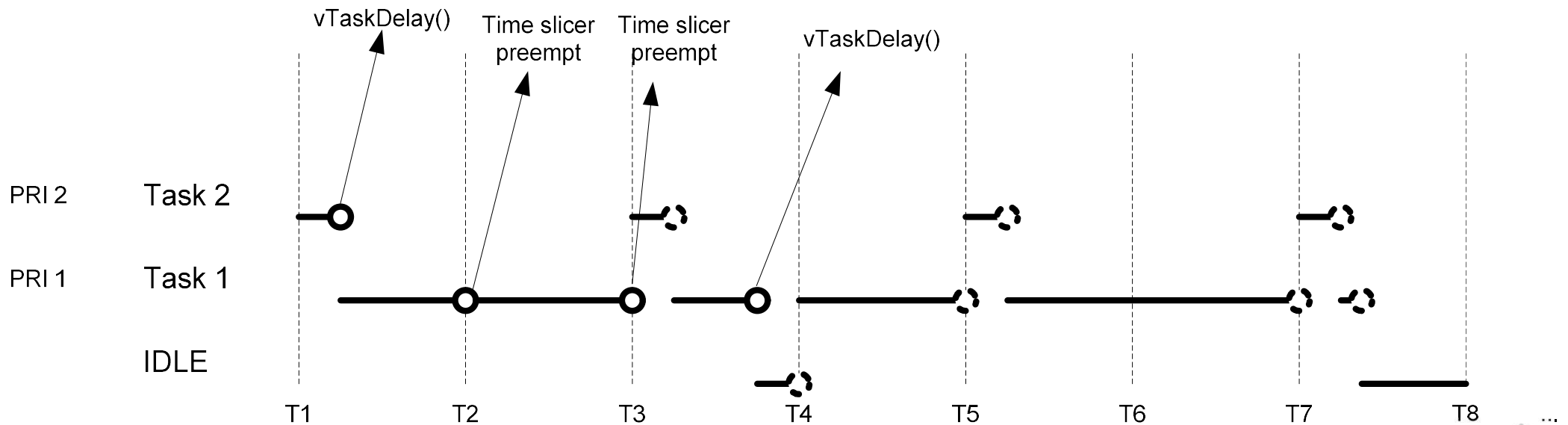
# Scheduling voorbeelden (5)

Task 1:

```
while (1)
{
    ... (lange operatie)
    vTaskDelay(1)
}
```

Task 2:

```
while (1)
{
    ...
    vTaskDelay(2)
}
```





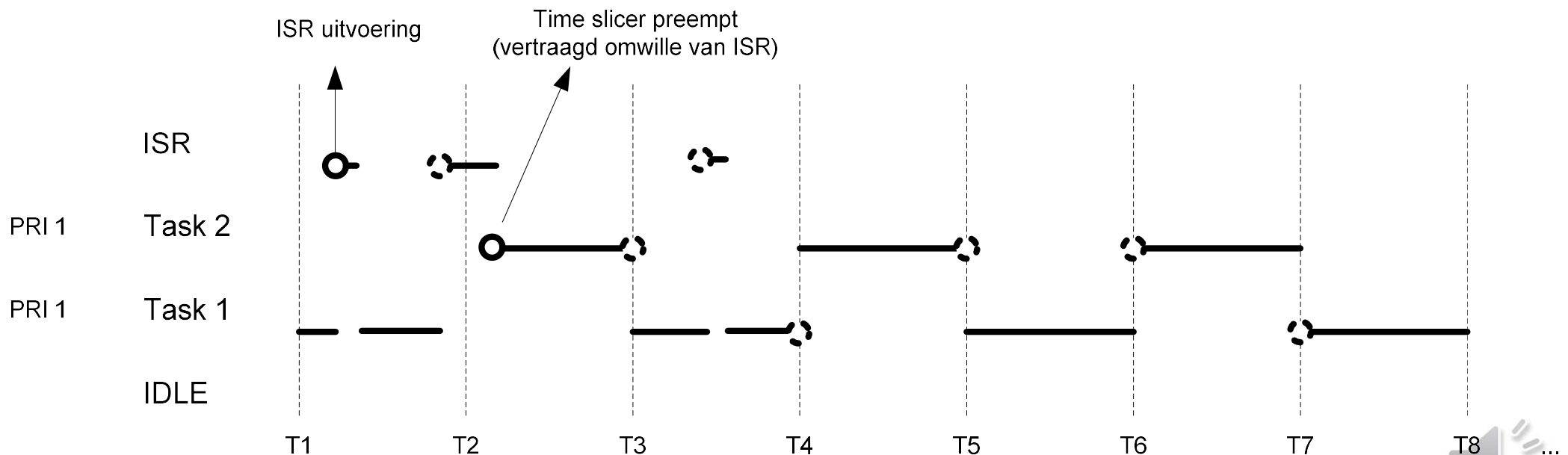
# Scheduling voorbeelden (6)

Task 1:

```
while (1)
{
    ...
}
```

Task 2:

```
while (1)
{
    ...
}
```



- Zie ref manual!
- Template: /tasks/TemplateTask.c, TemplateTask.h
- Aanmaken van tasks:

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        unsigned short usStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask );

void InitTemplateTask()
{
    xTaskCreate( WorkerTemplate, "template", configMINIMAL_STACK_SIZE,
                NULL, tskIDLE_PRIORITY+1, NULL );
}

static void WorkerTemplate(void *pvParameters)
{
    while(1)
    {
        ...
    }
}
```





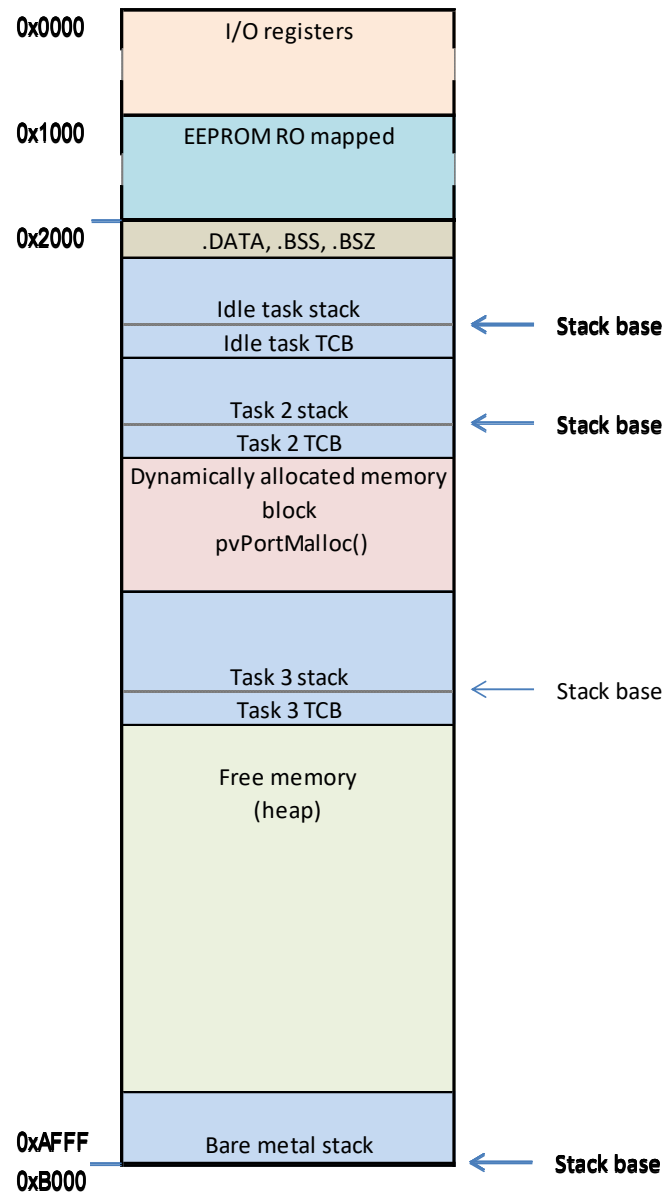
University of Antwerp  
Faculty of Applied  
Engineering

# Geheugenbeheer



# Geheugen organisatie

**ATXMEGA384C3 FreeRTOS memory map**



.DATA: initialized globals and statics  
.BSS: uninitialized globals and statics  
.BSZ: zero initialized globals and statics





University of Antwerp  
Faculty of Applied  
Engineering

# **Intertask communicatie:**

**Semaphores**

**Mutexes**

**Atomic operations**

**Queues**



# Atomic operations

- **Doel: stukken code atomair (=zonder onderbreking) uitvoeren**
- Onder FreeRTOS:
  - *taskENTER\_CRITICAL()*: Interrupts worden uitgeschakeld
  - *taskEXIT\_CRITICAL()*: Interrupts worden terug ingeschakeld
- → *Resultaat*:
  - Geheugenoperatie in de critical section kan niet onderbroken worden door ISR
  - Geen task switching tijdens de sectie
- Nadeel: real time gedrag wordt verstoord
- Korte critical sections: geen probleem.
- Semaphores / Mutexes / Queues maken intern gebruik van critical sections



# Atomic operations: voorbeeld

```
static uint8_t data[2];  
void TaskWorker1()  
{  
    uint8_t  
    while (1)  
    {  
        ...  
        taskENTER_CRITICAL();  
        data[0]=Sensor1Data;  
        data[1]=Sensor2Data;  
        taskEXIT_CRITICAL();  
        ...  
    }  
}  
void TaskWorker2()  
{  
    while (1)  
    {  
        ...  
        taskENTER_CRITICAL();  
        Sensor1Data=data[0];  
        Sensor2Data=data[1];  
        taskEXIT_CRITICAL();  
        ...  
    }  
}
```

Deze code wordt  
atomair uitgevoerd.

Deze code wordt  
atomair uitgevoerd.



- **Doel: gedeelde bron (geheugen, periferie, ...) beschermen tegen gelijktijdig gebruik**
- Mutex wordt aanzien als een speciaal type semafoor
- Aanmaken van mutex: *xSemaphoreCreateMutex()*
- Nemen van mutex: *xSemaphoreTake()*
- Geven van mutex: *xSemaphoreGive()*. Dit kan uitsluitend door de task die de mutex heeft gekregen!
- Indien in een ISR: *FromISR()* functievariant, zie ref manual!
- Mechanisme tegen prioriteitsinversie: priority inheritance  
<https://www.freertos.org/Real-time-embedded-RTOS-mutexes.html>



# Mutex: voorbeeld (1)

```
void TaskWorker1()  
{  
    while (1)  
    {  
        ...  
        xSemaphoreTake (ADCMutex,t);  
        | var1=GetADCValue(3,-1);  
        xSemaphoreGive (ADCMutex);  
        ...  
    }  
}  
void TaskWorker2()  
{  
    while (1)  
    {  
        ...  
        xSemaphoreTake (ADCMutex,t);  
        | var2=GetADCValue(4,-1);  
        xSemaphoreGive (ADCMutex);  
        ...  
    }  
}
```

```
int var1=GetADCValue(int PinPos,int PinNeg)  
{  
    int res;  
    ...  
    StartConversion();  
    WaitForConversionComplete();  
    res=GetResult();  
    ...  
    return res;  
}
```

Meestal portMAX\_DELAY =  $\infty$



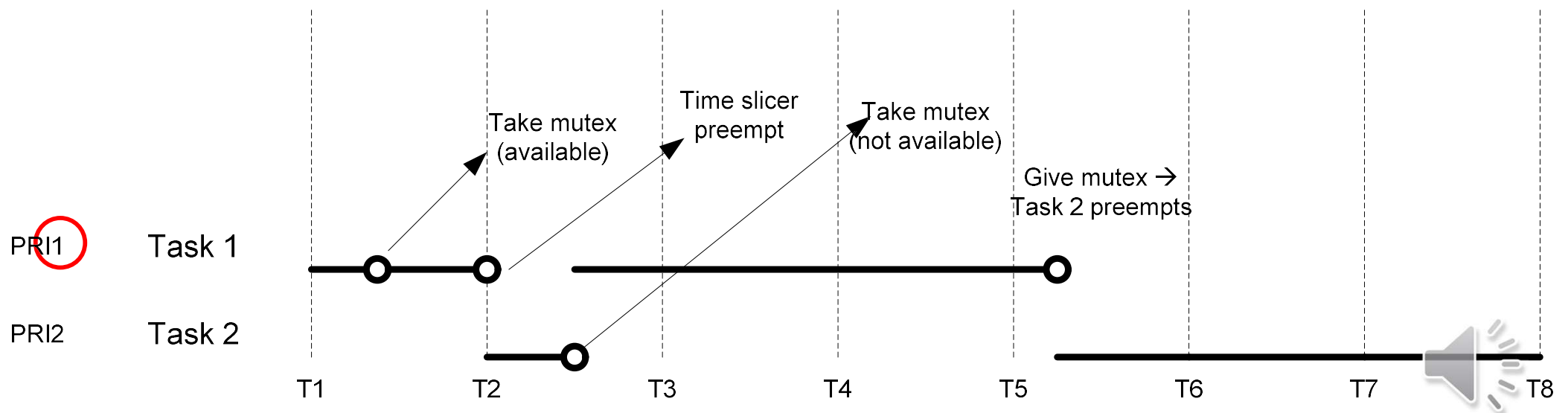
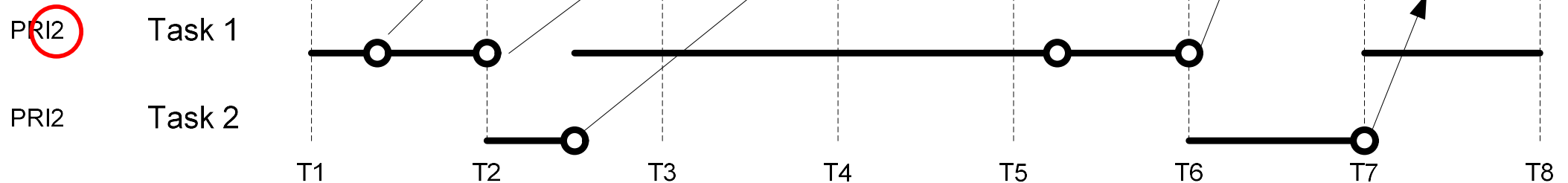
## Mutex: voorbeeld (2)

```
void TaskWorker1()  
{  
    while (1)  
    {  
        ...  
        var1=GetADCValue(3,-1);  
        ...  
    }  
}  
void TaskWorker2()  
{  
    while (1)  
    {  
        ...  
        var2=GetADCValue(4,-1);  
        ...  
    }  
}
```

```
int var1=GetADCValue(int PinPos,int PinNeg)  
{  
    int Result;  
    xSemaphoreTake(ADCMutex,t);  
    ...  
    StartConversion();  
    WaitForConversionComplete();  
    Result=GetResult();  
    ...  
    xSemaphoreGive(ADCMutex);  
    return Result;  
}
```



# Scheduling: mutex

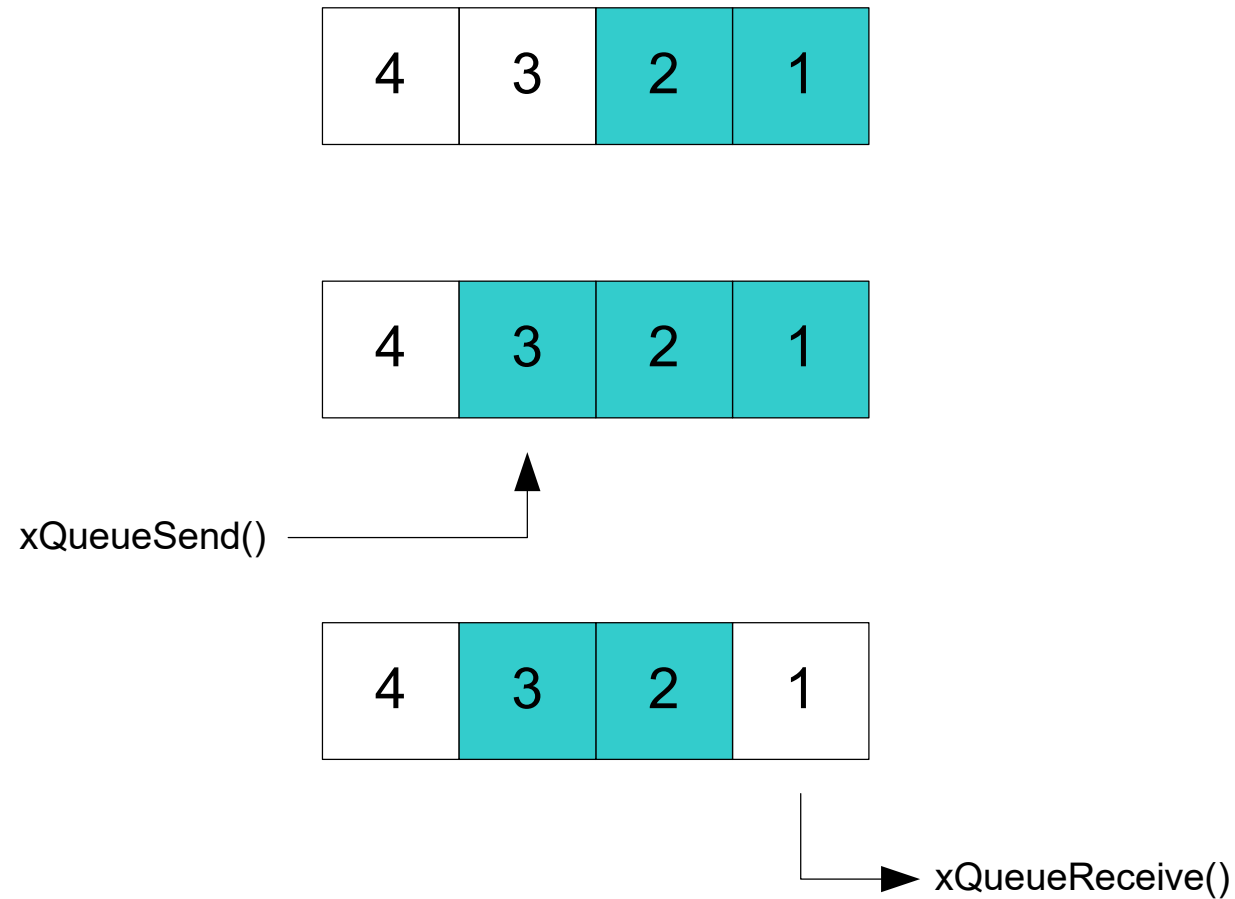


- **Doel: Datatransfer tussen tasks onderling of ISR en tasks**
- Eén of meerdere elementen in de queue
- Aanmaken van queue: `xQueueCreate()`
- Schrijven naar queue: `xQueueSend()`, `xQueueSendToFront()`, `xQueueOverwrite()`,
- Lezen uit queue: `xQueueReceive()`, `xQueuePeek()`
- Data wordt gekopieerd (niet by reference!). Dit gebeurt intern atomair d.m.v. critical section. Opgepast met grote data elementen!
- Indien in een ISR: ...*FromISR()* functievariant, zie ref manual!



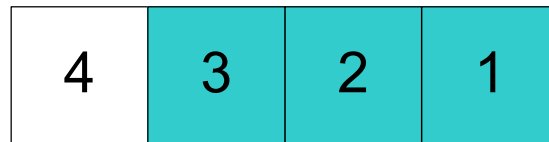
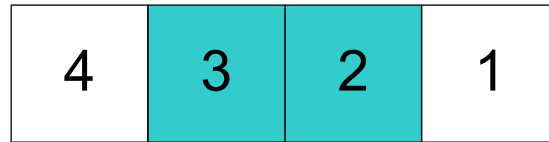
# Queue als FIFO

FIFO

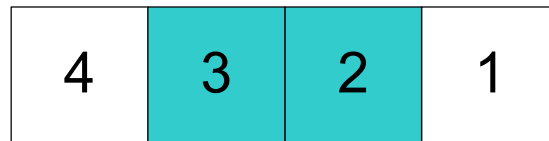


# Queue als LIFO

LIFO



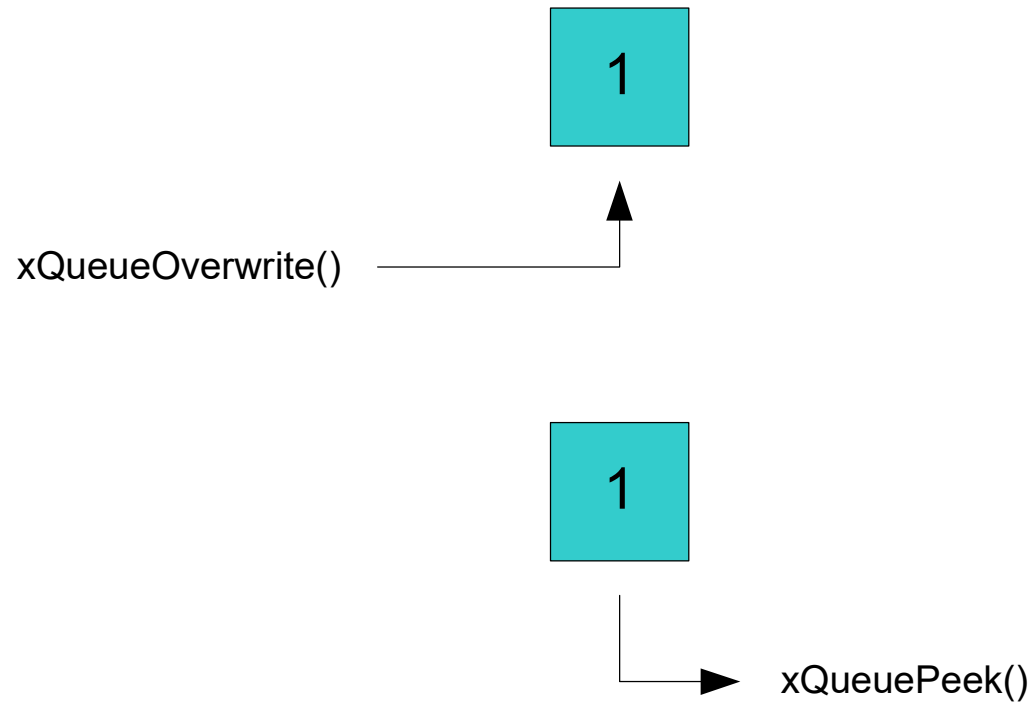
`xQueueSendToFront()`



`xQueueReceive()`

# Queue als shared variable

Shared  
variable



# Binary semaphore

- **Doel: synchroniseren van meerdere tasks / task met een ISR**
- Voorbeeld:
  - Sturen van een signal van één task naar een andere, bv om aan te geven dat een task moet afgesloten worden.
  - ADC Task wacht op voltooiing van een ADC conversie (binary semaphore)
- Aanmaken van semaphore: *xSemaphoreCreateBinary()*
- Nemen van semaphore : *xSemaphoreTake()*
- Geven van semaphore : *xSemaphoreGive()*
- Indien in een ISR: *FromISR()* functievariant



# Niet optimaal: ISR synch zonder semaphore

```
static int Result, ConversionComplete;
```

```
ISR(ADCA_CH0_vect)
{
    Result=GetResult();
    ConversionComplete=1;
}
```

```
void WaitForConversionComplete()
{
    while (!ConversionComplete);
    ConversionComplete=0;
}
```

# ISR synch met semaphore

```
static int Result, ConversionComplete;
```

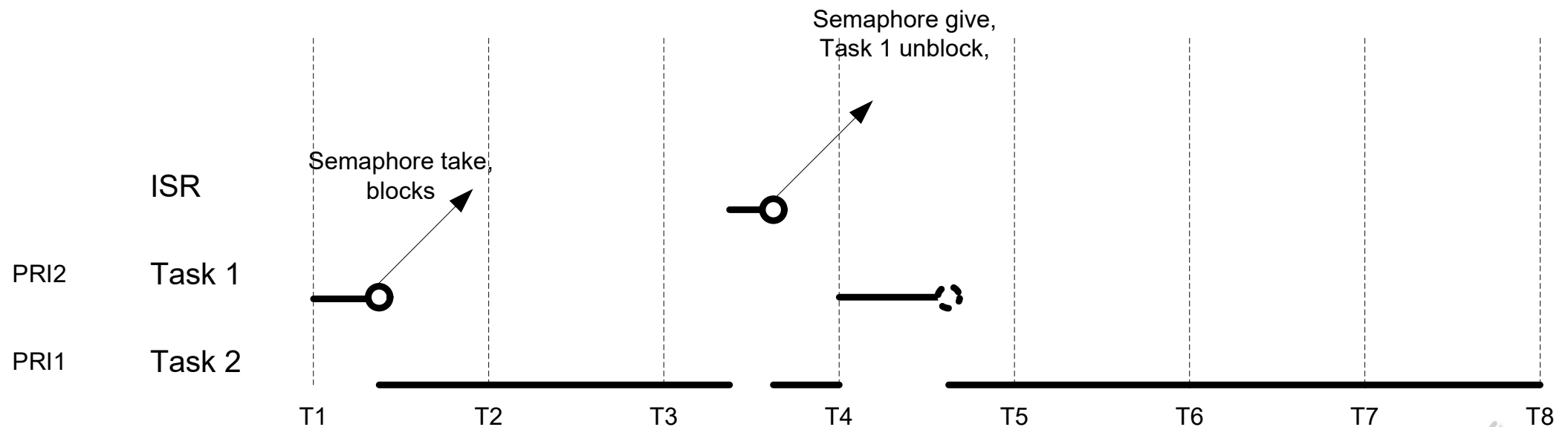
```
ISR(ADCA_CH0_vect)
```

```
{  
    Result=GetResult();  
    ...  
    xSemaphoreGiveFromISR(ADCReadySema, NULL);  
}
```

```
void WaitForConversionComplete()
```

```
{  
    xSemaphoreTake (ADCReadySema, t);  
}
```

# Scheduling: ISR semaphore synch: zonder TaskYield()



# ISR synch met semaphore:optimaal

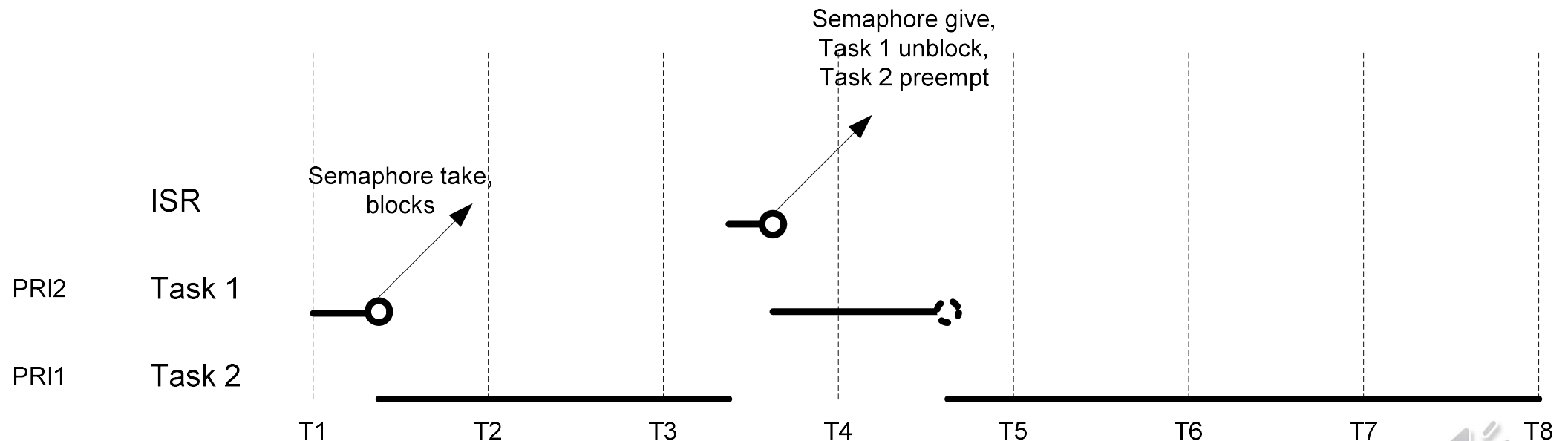
```
static int Result,ConversionComplete;

ISR(ADCA_CH0_vect)
{
    BaseType_t xHigherPriorityTaskWoken=pdFALSE;
    Result=GetResult();
    ...
    xSemaphoreGiveFromISR(ADCReadySema,& xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

void WaitForConversionComplete()
{
    xSemaphoreTake (ADCReadySema,t);
}
```

**Zoek uit in oefening 8 waarom dit beter is!**

# Scheduling: ISR semaphore synch: met TaskYield()



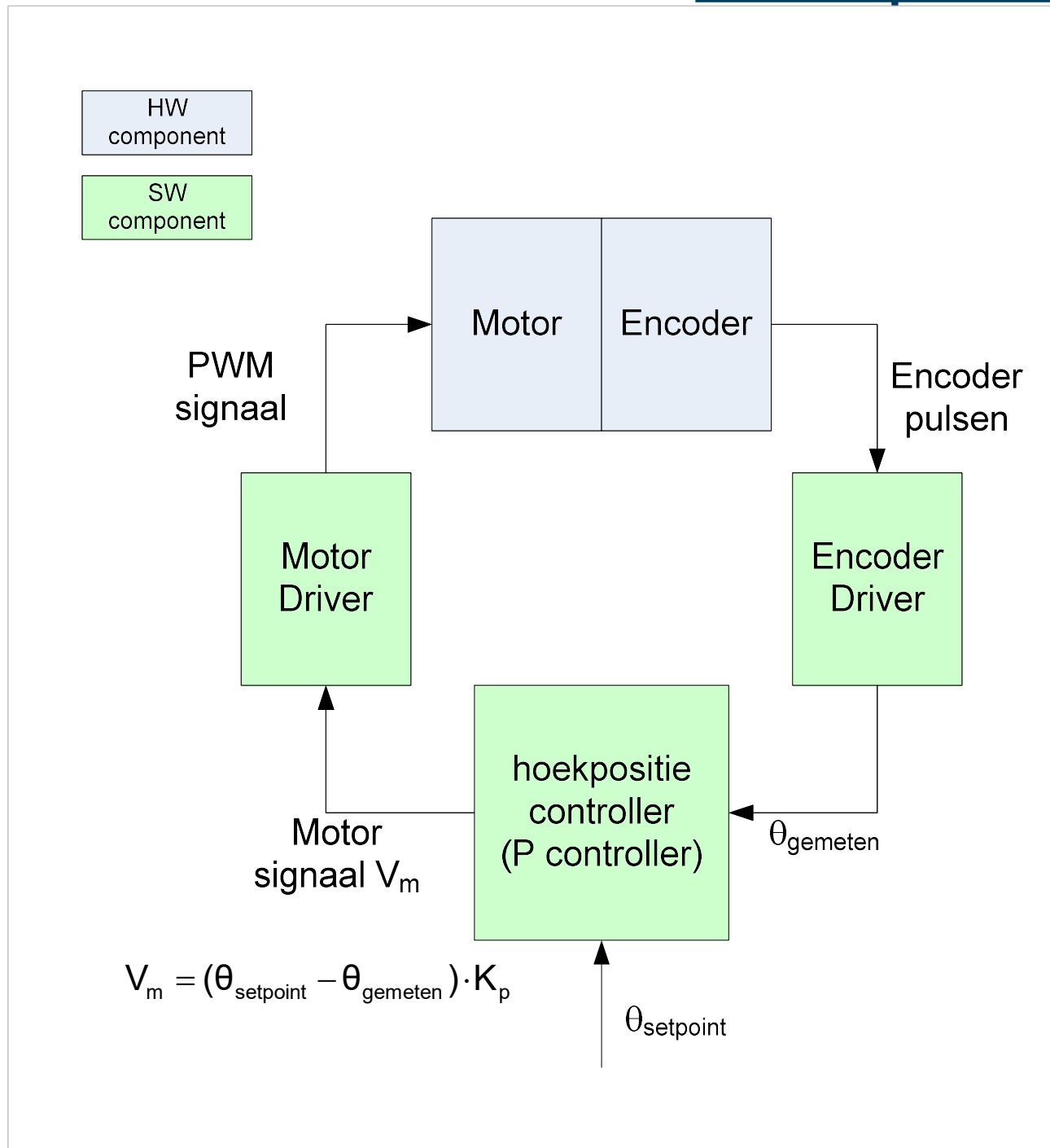


University of Antwerp

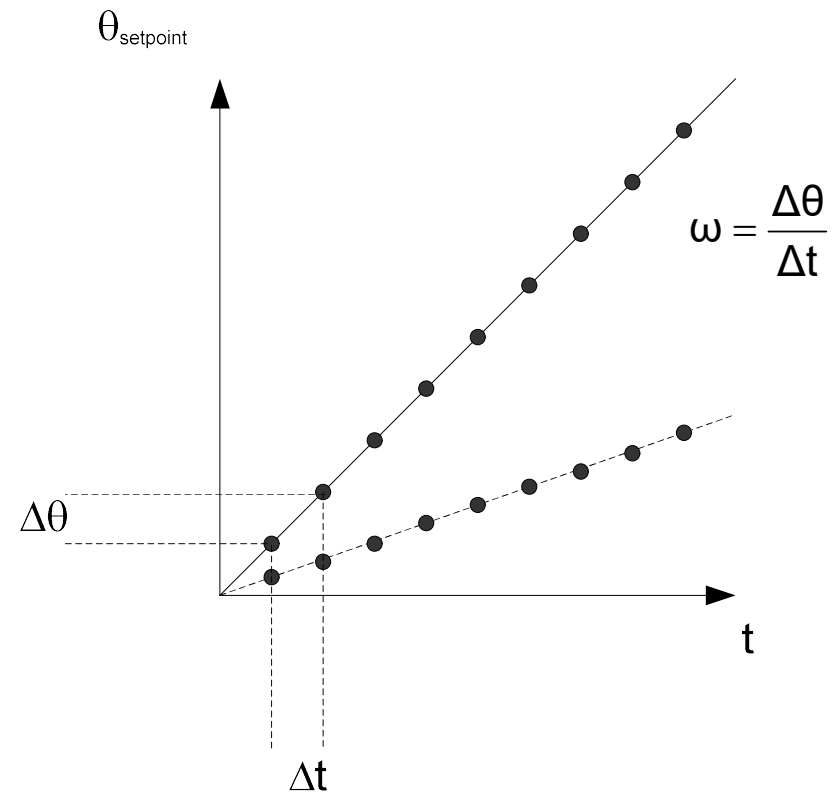
Faculty of Applied  
Engineering

# Oefening 10: motor hoekpositie- en snelheidscontroller

# Hoekpositiecontroller



# Hoeksnelheidscontroller



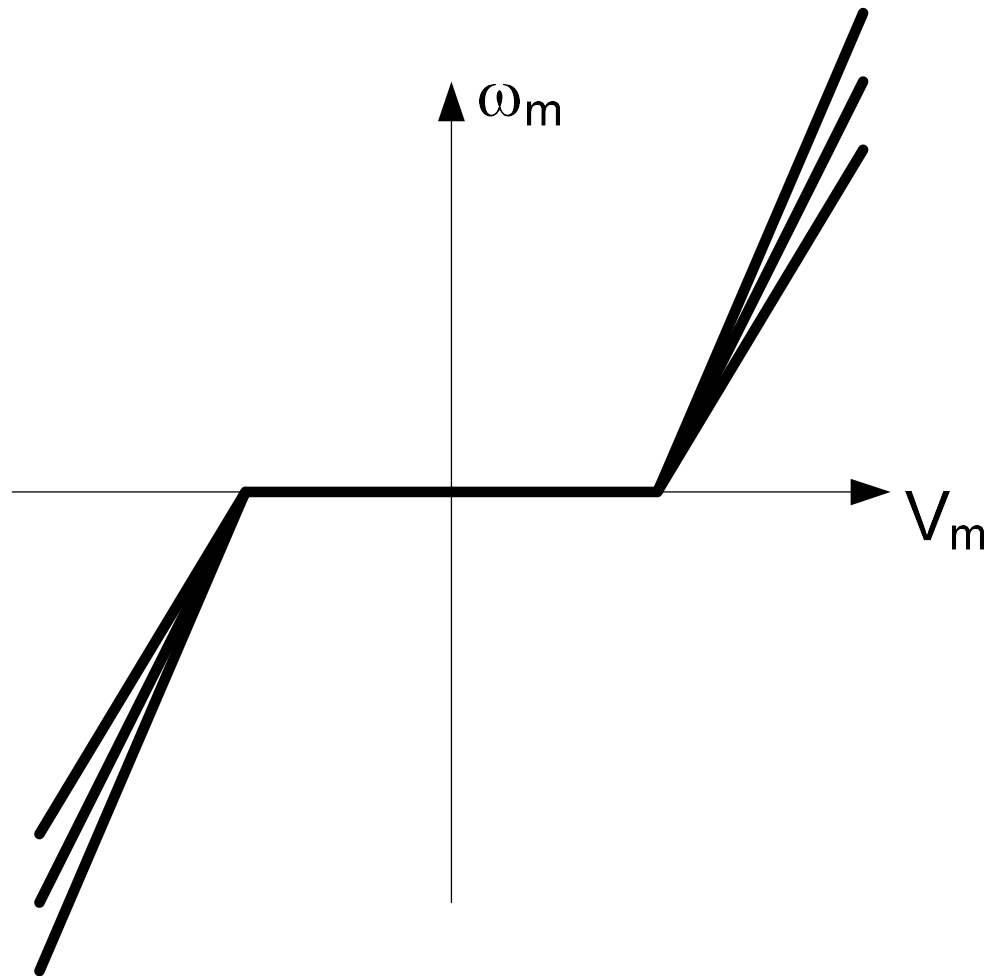


## Enkele praktische problemen

- Het te controleren system is niet-lineair (verband PWM duty cycle – motor snelheid) → eerst lineariseren
- Lage hoekresolutie v/d encoders → niet werken bij zeer lage hoeksnelheid

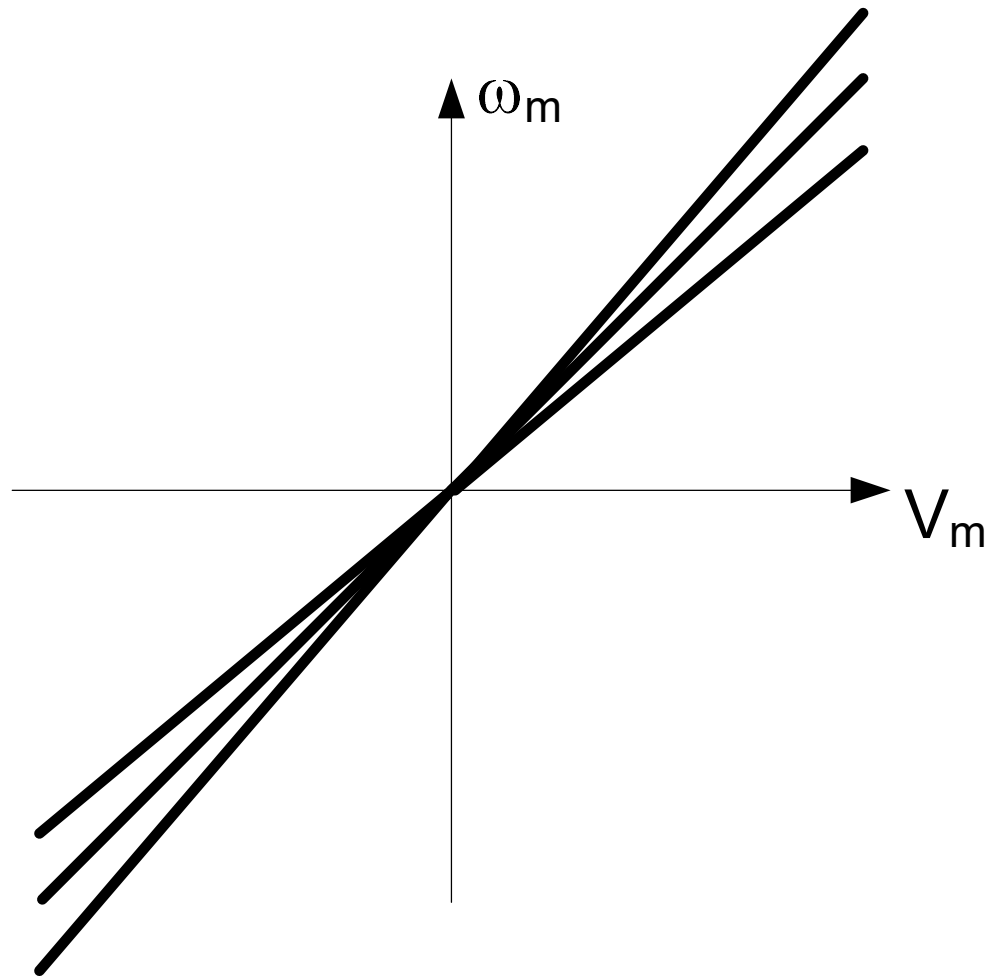
# Linearisatie van de motor aansturing

Responsie **voor** linearisatie



# Linearisatie van de motor aansturing

Responsie **na** linearisatie



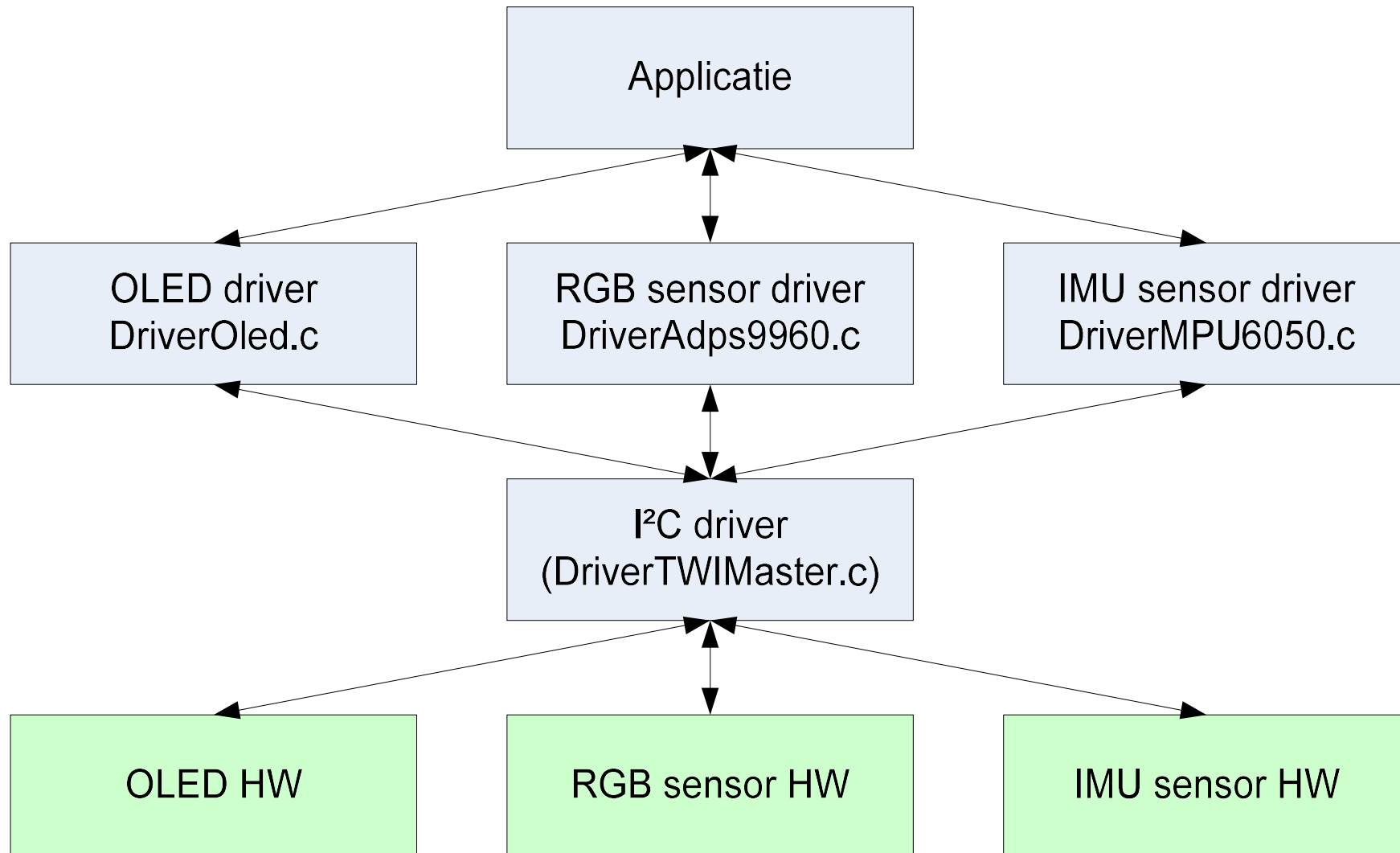


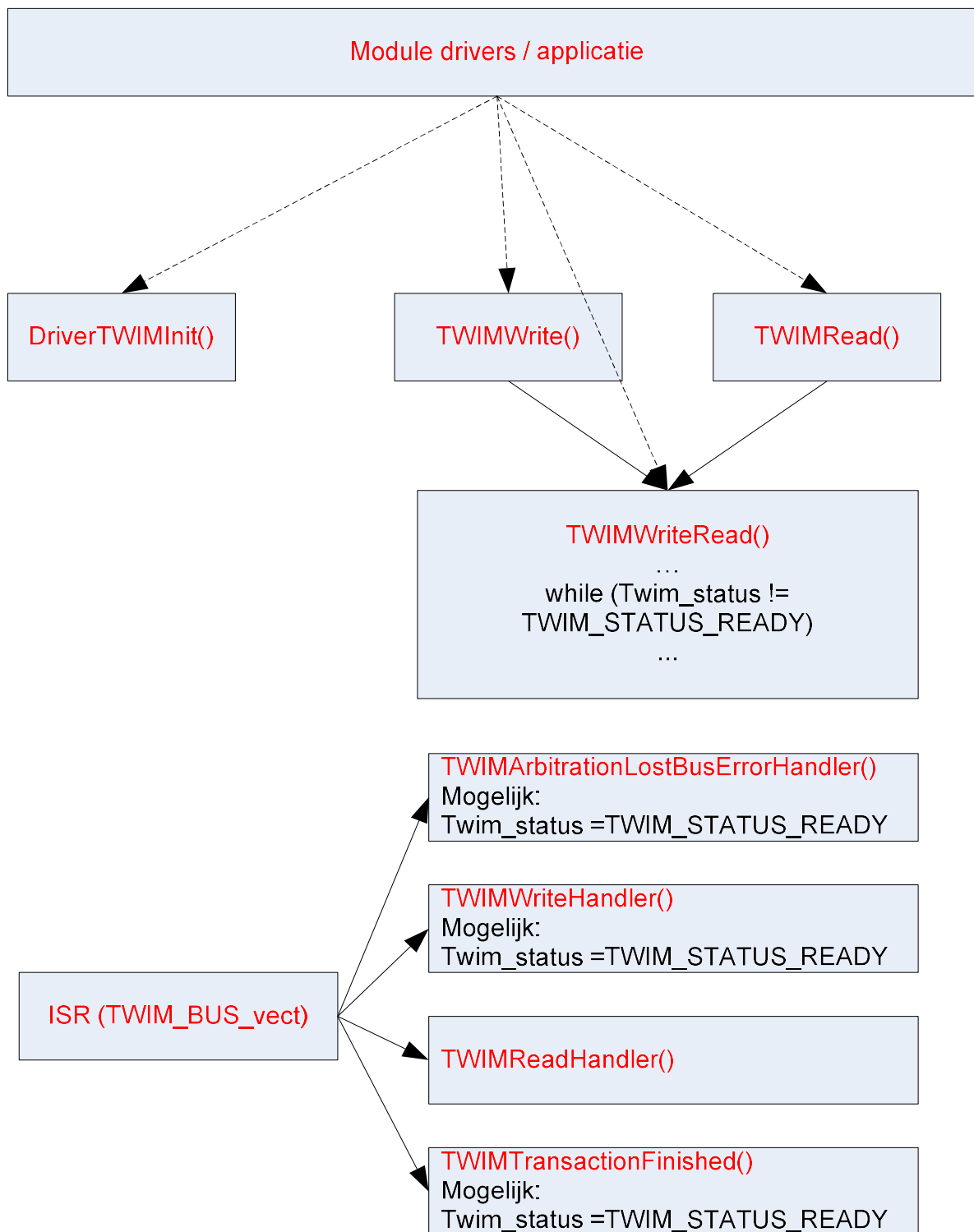
University of Antwerp  
Faculty of Applied  
Engineering

# Project 1: lokale user interface



# I<sup>2</sup>C driver structuur







University of Antwerp  
Faculty of Applied  
Engineering

# Project 2: oriëntatie meting d.m.v. een gyroscoop



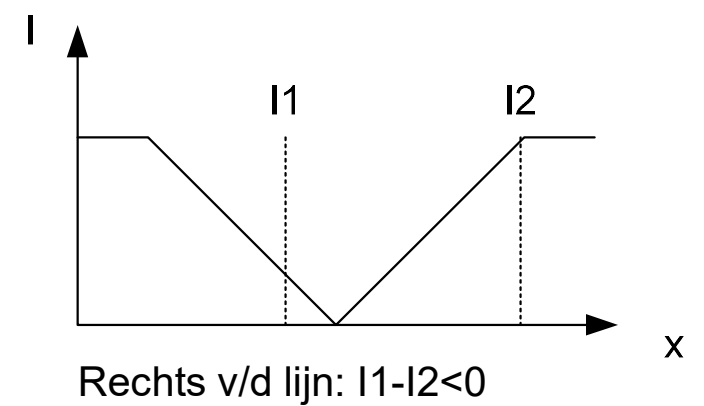
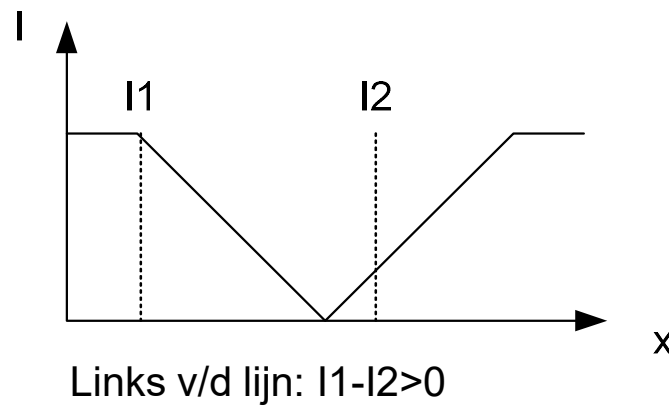
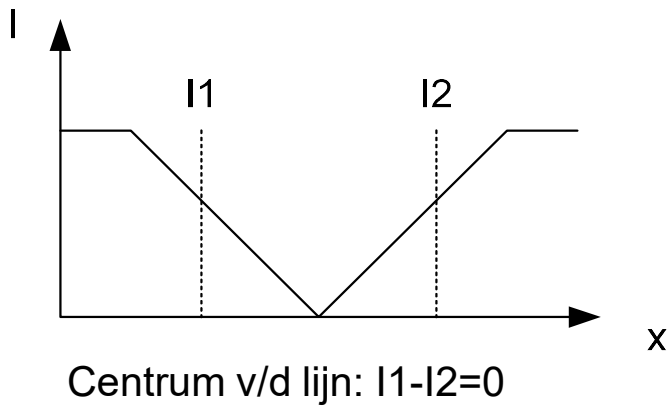
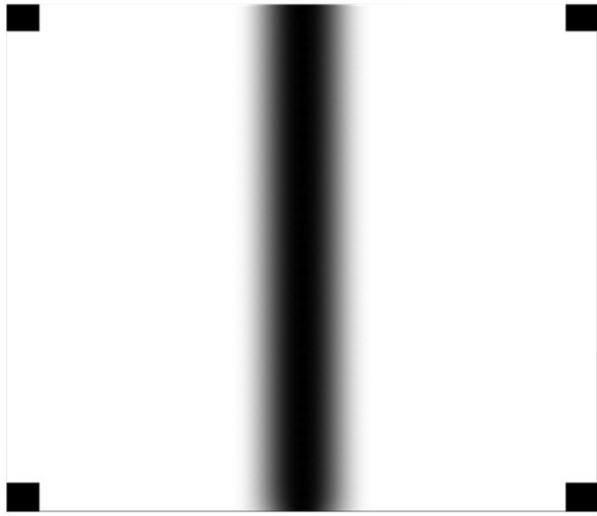
University of Antwerp

Faculty of Applied  
Engineering

# Project 3: lijnvolger



# Lijnvolger



$$V_{Mlinks} = V_{cst} + (I1 - I2) \cdot K_p$$
$$V_{Mrechts} = V_{cst} - (I1 - I2) \cdot K_p$$